



Academia de Studii Economice din București
Facultatea de Cibernetică, Statistică și Informatică Economică
Specializarea Informatică Economică

LUCRARE DE LICENȚĂ

Coordonator științific:
Conf. univ. dr. Mihai DOINEA

Absolvent:
Stela Stoimenova

București
2026



Academia de Studii Economice din București
Facultatea de Cibernetică, Statistică și Informatică Economică
Specializarea Informatică Economică

Interactive Web Platform for the Dance Industry

LUCRARE DE LICENȚĂ

Coordonator științific:
Conf. univ. dr. Mihai DOINEA

Absolvent:
Stela
Stoimenova

București
2026

Declarație privind originalitatea conținutului și asumarea răspunderii

Prin prezenta declar că rezultatele prezentate în această lucrare sunt în întregime rezultatul propriei mele creații cu excepția cazului în care se fac referiri la rezultatele altor autori. Confirm faptul că orice material folosit din alte surse (reviste, cărți și site-uri de internet) este în mod clar referit în lucrare și este indicat în lista de referințe bibliografice.

Table Of Contents

Contents

1. Introduction.....	1
1.1 Research Objectives and Questions.....	3
1.2 Thesis Organization and Progression.....	3
2. Digital platforms and the performing arts infrastructure challenge.....	4
2.1 Platform Economy Fundamentals and Network Effects.....	4
2.2 Event Management Systems and Requirements.....	5
2.3 Revenue Optimization Through Dynamic Pricing.....	7
2.4 Gamification and Engagement Mechanisms.....	9
2.5 Role-Based Access Control and Security Architecture.....	11
2.6 REST Architecture and Horizontal Scalability.....	12
2.7 Digital Transformation in Performing Arts.....	14
3. Technologies and Methods.....	16
RAD with Agile Iteration.....	16
React.js.....	17
Node.js and Epress.js.....	17
PostgreSQL with Prisma ORM.....	18
Zod.....	18
4. Interactive Web Platform Architecture and Data Model for the Dancing Industry.....	20
Data Model.....	23
5. Implementation and Deployment of FOMO.....	27
Deployment Infrastructure.....	30
Authentication Flow and Security Implementation.....	30
Ticket Purchase Transaction Implementation.....	30
6. Results, Evaluation, and evidence.....	32
Research Question Validation.....	33
Decision Navigating.....	34
Limitations and Future Scaling Constraints.....	34
Conclusions and Contribution Significance.....	35
Bibliography.....	37
Anexa 1 - List of Figures.....	40
Anexa 2 - List of Tables.....	40
Anexa 3 - List of Acronyms.....	40
Anexa 4 - REST API Endpoints Summary.....	42

1. Introduction

Digital transformation has reshaped nearly every economic sector globally from retail to finance to entertainment. The performing arts, particularly dance, experienced this shift profoundly. Artists and choreographers once relied on local networks and word-of-mouth. Today they promote through Instagram, TikTok, YouTube [1]. Events are discovered online, audiences connect with performers they'd never encounter otherwise, yet this visibility masks a fundamental problem which is linked to the infrastructure supporting dance events hasn't evolved at the same pace.

FOMO "Fear of Missing Out" is a platform designed to solve this mismatch. It provides dancers, studios, and choreographers with integrated tools for event creation, ticket sales, audience engagement, and sustainable business models rather than juggling separate tools (Instagram for discovery, PayPal for payments, email for communication), FOMO consolidates everything. The platform applies platform economics principles, modern web technology, and behavioral psychology to create an ecosystem where dancers profit from events and audiences find performances effortlessly.

Being embedded in this industry gave me direct insight into the challenges artists face. Workshops, competitions, castings, classes, artistic projects, all get promoted through social media, yet a critical limitation emerges when you examine how events actually operate.

Social media reaches large audiences quickly that's undeniable, but it wasn't designed for event management. Ticket sales, participant coordination, financial tracking are happening separately often through email chains, spreadsheets, or informal communication [2]. Visibility exists without the infrastructure to support it. Eventbrite, Meetup, and Ticketmaster handle ticketing and event logistics, but they're generic platforms indifferent to dance. Niche dance websites and Facebook groups share some event information, yet they offer no performer portfolios, no loyalty systems, no revenue optimization tools which means nothing is integrated.

No unified platform exists that combines social discovery, event posting, ticketing, and economic mechanisms designed specifically for dancers..

The market gap in the dancing industry creates what I call "blind spots." Dancers struggle to find opportunities beyond their immediate circles, studios spend excessive time on administrative overhead when they should focus on choreography and performance. During conversations with choreographers and performers in Bulgaria and Romania, I repeatedly heard the same complaint: artists must invest enormous energy promoting themselves on Instagram which is a

platform built for socializing, not event discovery in order to reach new audiences. Without an established social media presence or influencer status, emerging dancers get invisible. The core problem runs deeper than visibility. Instagram demands constant attention to metrics, aesthetics, follower counts pulling energy away from the actual choreography, performance, creative development. Getting invited to paid gigs requires prior reputation or connections, locking out new talent. Several specific gaps define the market opportunity.

Event discovery remains fragmented. Dancers searching for workshops, competitions, or performances must check multiple websites, social media accounts, and local Facebook groups. Information scatters across platforms, reach stays limited to local audiences.

Ticketing creates friction. Manual registration, email confirmations, external payment processors are all add steps for attendees. Organizers can't implement dynamic pricing, tiered tickets, or loyalty programs, revenue caps at fixed ticket prices.

Engagement drops after events. Once performances end, organizers have no mechanism to maintain relationships or encourage return attendance. Audiences must be re-acquired from scratch for each new event.

Data remains unavailable. Organizers don't understand their audiences like demographics, preferences, attendance patterns. Without insights, they can't optimize future events or justify pricing strategies.

The digital community doesn't exist. Dancers, choreographers, studios operate in isolation. No unified space for networking, collaboration, or shared learning.

FOMO addresses all of these. It's a specialized digital environment where performers create portfolios, post events, sell tickets, earn loyalty points, and track attendee behavior on a single platform. Studios and individual dancers can search for talents, dancers can promote their work directly to industry professionals. The platform implements three proven economic mechanisms. Platform economics literature demonstrates that two-sided markets connecting providers (organizers) and consumers (dancers) generate network effects: more events attract more attendees, more attendees attract more organizers, creating self-reinforcing growth [3]. Dynamic pricing research indicates that adjusting ticket prices based on real-time demand and inventory can increase organizer revenue 8 - 12% versus static pricing without degrading customer fairness perception, provided pricing logic remains transparent [4][9]. Gamification, specifically loyalty point systems directly tied to ticket purchases and redemption, boosts repeat attendance by 20 - 30%, as documented through examination of transaction-based incentive structures [6][12].

1.1 Research Objectives and Questions

This thesis centers on one primary research question with several supporting inquiries:

To what extent can an integrated digital platform employing dynamic pricing, role-based access control, and transparent incentive mechanisms enhance event discoverability, optimize organizer revenue, and establish community cohesion within the dance sector?

The scope intentionally constrains itself to core transactional features: user registration, event lifecycle management (creation through closure), ticket inventory and purchase workflows, financial reconciliation, and basic community features (following, reviews, messaging). Advanced features like livestreaming, machine learning recommendations, native mobile applications, multiple payment processor integration, multi-currency support are falling outside current scope. Payment processing uses Stripe's sandbox environment (simulated transactions, not live processing). Dynamic pricing employs deterministic inventory-driven algorithms rather than machine learning demand forecasting. These limitations are acknowledged, they do not compromise the proof-of-concept validity

Key objectives guide the implementation. The platform must be scalable and user-friendly, tailored to dance industry needs. Event organizers need minimal friction when creating, promoting, and managing events. Attendees require advanced search, filtering, and recommendation capabilities to discover relevant performances. Dynamic pricing mechanisms [4] should optimize revenue while maintaining perceived fairness [5]. Points, badges, tiers, rewards [6] must increase engagement. Security demands role-based authorization [7] protecting user data and payment information. Analytics should provide organizers comprehensive insights into attendee behavior. Finally, community features must enable networking and collaboration among dance professionals.

1.2 Thesis Organization and Progression

Chapter 2 situates the research within existing literature, examining platform economics, event management system design, revenue optimization through dynamic pricing, user engagement through gamification, security through role-based access control, and technical architecture supporting scalability. Chapter 3 details the development methodology and technology stack selection, justifying each technical choice against specific requirements. Chapter 4 decomposes the system architecture, explaining the three-tier design (presentation, business logic, persistence) and the data model's 14 interconnected entities. Chapter 5 covers implementation details: database setup, API endpoints, frontend components, and deployment infrastructure. Chapter 6 presents results and evaluation, validating achievement against research questions and documenting technical performance, synthesizing findings, discussing implications for

digital transformation in performing arts, and proposes credible future research directions grounded in architectural constraints

2. Digital platforms and the performing arts infrastructure challenge

Literature review focuses on industry best practices relevant to FOMO: platform economics, event management systems, revenue optimization, user engagement, security, and performing arts digitalization.

2.1 Platform Economy Fundamentals and Network Effects

A platform economy emerges when digital systems connect distinct stakeholder groups to facilitate transactions and value exchange. Rochet and Tirole [3] established that digital marketplaces differ fundamentally from traditional systems. They create value through network effects. More users joining makes the platform more valuable for everyone. Direct communication between parties reduces intermediaries, transaction costs drop. FOMO exemplifies this structure. Event organizers and dancers constitute separate demand-side and supply-side markets. Without FOMO, organizers struggle to reach audiences beyond direct networks, and dancers struggle to discover opportunities beyond local venues and social media happenstance.

Successful platforms share several characteristics. Two-sided (or multi-sided) markets connect buyers and sellers, providers and consumers are two distinct groups that need each other [3]. FOMO operates in a simple way meaning that event organizers are on one side, dance enthusiasts are on the other one. Network effects compound value non-linearly. Each new organizer joining FOMO increases the platform's appeal to dancers (more event choices). Each new dancer strengthens the platform's value proposition for organizers (larger potential audience). This circular reinforcement distinguishes platforms from traditional software. An accounting system benefits one organization, but a marketplace strengthens with every participant. The effect manifests directly in FOMO's architecture where the recommendation algorithm surfaces personalized events as the user base expands, community features like following, reviews, ratings become more valuable as more dancers participate and data analytics for organizers improve as transaction volume accumulates. Transaction costs diminish when platforms automate discovery, matching, and payment processing [3]. FOMO replaces manual event searches with filtering, recommendation algorithms, and consolidated ticketing. Trust mechanisms matter as well as rating systems, reviews, and verified credentials allow strangers to transact confidently. FOMO builds this through organizer profiles displaying past event ratings and attendee reviews. Ecosystem approach extends platform value. Third-party

integrations (Stripe for payments, SendGrid for email, analytics tools) let organizers extend functionality without building it themselves.

Radonjic-Simic and Pfisterer [8] extended platform theory beyond these fundamentals. They proposed that traditional centralized platforms excel at standardized transactions but struggle with customization and distributed governance. They are proposing "Distributed Market Spaces" which are decentralized, self-organizing markets at internet scale. This model applies directly to dance. FOMO implements it through flexible event creation (each choreographer customizes their own event type), customizable ticket structures, organizer-specific feature sets, and community-driven governance allowing the platform to adapt to diverse performers and audiences rather than imposing standardized models.

Transaction cost reduction constitutes the second platform mechanism. Pre-FOMO, studios promoted events through Instagram posts, received registrations via direct messages (timeintensive), collected payment through bank transfers (slow, error-prone), tracked attendance manually, and possessed no systematic data about attendee preferences or behavior. Each step required manual intervention and created friction. FOMO collapses these into automated workflows:

event listing -> discovery -> ticket purchase -> payment processing -> attendance confirmation -> data aggregation

The friction reduction directly translates into conversion increases, Sykimte [10] documents that simplified checkout flows (*select ticket type -> enter details -> confirm payment*) boost conversion 25–40% versus multi-step forms requiring external redirects. 10 Trust mechanisms enable transactions between strangers. Organizers' profiles display past event ratings and attendee reviews, establishing credibility through social proof. Dancers see instructor credentials (VERIFIED status on CVs), reducing risk of booking low-quality instruction. This trust infrastructure matters empirically. Platforms without transparent verification mechanisms experience lower conversion and higher refund requests [8].

2.2 Event Management Systems and Requirements

Event management used to be straightforward with schedule, venue, and register attendees. Today it's much more complex. Organizations now juggle event creation, ticketing, marketing, communication, analytics, and mobile platforms simultaneously. Gokul et al. [2] point out that this integration across seven functional domains determines whether systems actually succeed operationally.

When organizers set up an event, they need more than just a date and location. They input detailed descriptions, images, capacity limits, pricing tiers [2] and for the dance part they add dance styles, skill requirements, and venue specs like stage size or lighting. Dance events demand specificity that generic platforms don't support. FOMO sidesteps this by letting choreographers and studios configure events using dance-specific templates rather than forcing their work into one-size-fits-all boxes. Failure in any single domain cascades for example a platform with brilliant analytics but poor mobile support loses users because of discovery where 70 to 85% of event browsing occurs which proves unusability [2] [12].

Dance events demand domain specificity. Creating a generic event requires inputting basic data: date, title, description, capacity, single ticket price. Creating a dance event demands choreography-specific inputs: applicable dance styles (contemporary, ballet, hip-hop, etc.), required experience level (beginner, intermediate, advanced), venue specifications affecting performance (stage size, available lighting), instructor credentials, accessibility accommodations (wheelchair access, ASL interpretation, sensory-friendly options), and often recurring class structures with membership tiers. Generic platforms force dance organizers to improvise work around (using description fields for style, picking arbitrary skill levels, hoping attendees understand requirements from narrative). FOMO integrates these as first-class data types, enabling precise filtering and discovery.

Promotion changed too. Rather than depending on email blasts, systems now coordinate email, social media, targeted advertising, and recommendation algorithms based on attendee history [2]. For dancers, visibility is everything, yet they mostly rely on Instagram, a platform built for socializing, not event discovery. FOMO addresses this by letting artists create portfolios showcasing past performances, reviews, ratings which helps audiences find creators directly instead of waiting for algorithmic suggestions. After events end, most platforms abandon attendees. No communication, no follow-up, nothing to keep them engaged for future events. Effective systems send automated reminders in 7 days/1 day/1 hour before, then thank-you emails and post-event surveys [2]. This cuts no-shows by 15 to 25% and increases attendee satisfaction because expectations stay clear throughout.

Data is one of the successful tools for development. Organizers should see ticket sales by type, revenue, sell-through rate, attendee demographics, financial reports like gross revenue, fees, net income, and engagement metrics such as email opens, attendance rates, survey responses [2]. Without these insights, organizers can't improve future events or understand their audiences. Effective systems compare current event performance against past ones to reveal patterns and opportunities. Loyalty systems increase attendee lifetime value measurably, where

repeat attendance generates redeemable value[6][12]. FOMO implements automated reminders tied to event timelines and transaction-based loyalty points tied directly to ticket purchases, making repeat attendance financially beneficial to dancers. Data availability separates thriving events from stagnant ones. Organizers lacking insights cannot optimize. With analytics, organizers understand which dance styles drive attendance, which pricing points maximize revenue, which promotional channels convert best, which demographic segments show the highest lifetime value. This data informs future event design, pricing strategy, and marketing allocation. Absent this feedback loop, organizers repeat whatever worked last time, unable to evolve. FOMO surfaces organizer dashboards displaying ticket sales velocity, how quickly tickets sold in the first hour/day/week, demographic summaries of attendees, revenue breakdowns by ticket tier, and engagement metrics (email open rates, review count, follower growth).

Mobile traffic now drives 70 to 85% of visits [2], so responsive design, mobile ticketing with QR codes, location-based discovery, and frictionless checkout are non-negotiable. Tan et al. [12] demonstrated mobile optimization increases conversion up 30 to 45% versus desktop-only sites. FOMO prioritizes mobile experiences.

Security can't be an afterthought. Digital systems must enforce role-based access control since it is logical for organizers to see only their own financial data, attendees can't modify event details [7]. Encryption protects passwords and payment information, PCI-DSS compliance ensures secure payment processing through providers like Stripe [16]. Audit logs track every action through the login, payment and data access, for forensic analysis if breaches occur [7].

Finally, dance-specific requirements go beyond ticketing. Dancers need digital portfolios showcasing past work. Attendees need to filter by dance style and skill level. Organizers need to specify accessibility such as wheelchair access, ASL interpretation, sensory-friendly shows. Platforms should connect dancers, choreographers, and organizers for collaboration [10]. Instructors should have verified credentials.

2.3 Revenue Optimization Through Dynamic Pricing

Revenue optimization separates successful event platforms from mediocre ones. Static ticket prices, meaning same price regardless of demand or timing, leave money on the table. Elmaghraby and Keskinocak [4] established that dynamic pricing is adjusting ticket prices based on demand, inventory, and market conditions, usually significantly increasing revenue while optimizing inventory utilization. Airlines pioneered this as well as hotels, entertainment venues, and sports teams adopted it systematically. The logic is pretty straightforward, when demand

spikes, raise prices and when demand weakens, lower the prices to encourage sales and fill remaining inventory. Razumovskiy, Karenin, and Safro [9] advanced this research by examining how pricing accuracy affects revenue in event ticketing specifically. They found that misspecifying demand patterns is leading to failing accounts regarding customers' willingness to pay changes as event dates approach which causes costs organizers 0.42% to over 1% of potential revenue. Most critically, organizers who ignore late-demand surges (the rush to buy tickets days before events) lose substantially more. This precision matters enormously in dance particularly because of the weekend performances, competitions, workshops with all experience predictable demand curves. Understanding these patterns, then pricing accordingly, directly impacts organizer income.

Several variables shape dynamic pricing decisions but remaining inventory is primary. The available seats decrease, prices rise to maximize revenue from scarce stock resulting in filling the last 10% of capacity at premium prices rather than discounting. Time until the event matters equally, as performance dates approach, organizer willingness to discount typically increases (if sales lag) or prices rise further (if demand accelerates). Historical patterns from similar past events inform pricing and an example would be a Saturday jazz workshop attended by 80 last year suggests pricing similar workshops around that expected volume. Real-time booking velocity, meaning how fast tickets sell in the first hour, first day, first week typically reveals whether demand is unexpectedly strong or weak. Organizers adjust prices instantly based on this signal. Competitor pricing prevents isolation, since if your studio charges \$40 for a hip-hop class when three competitors charge \$25 to \$30, participation drops regardless of quality. Finally, customer segments have different price sensitivities [4]: professional dancers might pay \$50 for a masterclass but students might not. Strategic segmentation through tiered tickets (standard, VIP, group rates) captures value across the entire customer base. Şahin and Erol [11] designed a dynamic pricing model for sports events using fuzzy logic to forecast demand, demonstrating 8 to 9% revenue gains compared to static pricing. While their research focused on soccer, the methodology applies directly to dance events. The key insight is a demand forecasting which drives everything while predicting how many people will buy at specific prices. Without accurate forecasting, pricing decisions become guesses and with it, organizers systematically extract more value. Yet dynamic pricing creates a perception problem. Customers discover they paid \$20 yesterday while someone else paid \$30 today for identical seats, generating resentment. Fairness matters as Elmaghraby and Keskinocak [4] note that price transparency and perceived fairness are critical to customer acceptance. Organizers who implement dynamic pricing without explaining the logic like "prices increase as inventory depletes, you benefit by purchasing early" risk damaging trust. The solution isn't abandoning

dynamic pricing but embracing transparency. Airlines, Uber, and concert platforms overcome this resistance by clearly showing customers why prices vary.

FOMO implements intelligent, transparent dynamic pricing. Rather than complex algorithms, organizers can't understand. The system lets choreographers and studios set base prices and simple rules: "increase 10% when 75% of capacity sells," "decrease 15% if seven days remain with fewer than 50% sold," "offer 20% early-bird discount for purchases more than 30 days in advance." The system calculates optimal prices in real-time, organizers see the logic behind every price change. Attendees see the same transparency: ticket prices display remaining inventory ("only 8 seats left at this price") and time-until-event messaging ("early-bird pricing ends in 5 days"). This approach combines Razumovskiy's precision with Şahin and Erol's forecasting insights while prioritizing the customer fairness Elmaghraby and Keskinocak identified as essential. The result is: organizers capture 8 to 12% additional revenue without sacrificing customer trust or alienating price-sensitive attendees who benefit from early-purchase discounts.

Dynamic pricing becomes especially powerful in dance when combined with event characteristics. Saturday performances attract families, weeknight workshops attract professionals. Recurring events like weekly classes sustain demand differently than one-time competitions. FOMO's pricing rules adapt to these patterns: high-demand Saturday performances escalate prices rapidly as capacity fills, on the other side less-popular Thursday classes discount aggressively to fill seats. Organizers focus on choreography and performance while the system optimizes pricing.

2.4 Gamification and Engagement Mechanisms

User retention in ticketing platforms hinges on reducing friction. Most platforms rely on email reminders or push notifications, passive mechanics that don't address why attendees should return. Koneti et al. [6] document that loyalty rewards, when well-designed, substantially improve repeat attendance and customer satisfaction. The method is simple: if users perceive direct financial benefit from returning, they come back more frequently. FOMO implements this through a transaction-based loyalty system where attendance directly translates to future savings [6]. Rather than abstract gamification such as badges, leaderboards, tier progression, the platform uses a simple earning-and-redemption formula grounded in direct financial incentive [12].

The procedure itself is transparent. When a user makes a purchase they earn 5% of the ticket price as points on every spending, meaning 10 points are equal to €0.10 discount (exchange rate: 10 pts -> 1 cent). Points are applied automatically by checking "Use Points" at checkout. No complexity, no variable bonus multipliers. This simplicity matters since customers understand

immediately how value accrues. Purchase a \$120 workshop: earn 600 points. Next ticket purchase, those points redeem: 600 points becomes a \$6 discount on a subsequent event [6]. The math is obvious, avoiding the perception problem that Elmaghraby and Keskinocak [4] identified as critical to customer acceptance, users see exactly why they're being rewarded and how much it's worth.

Redemption applies a natural ceiling. A user cannot discount any ticket by more than 15% using loyalty points alone; the platform maintains this cap to preserve organizer revenue while still rewarding loyalty [4]. For a \$50 ticket, maximum points discount is \$7.50 (750 points). For \$200 performance, maximum is \$30 (3,000 points). This constraint prevents the system from becoming a subsidy mechanism. Loyalty rewards consistency and commitment but doesn't undermine ticket pricing strategy. Transparency forms the psychological foundation. The system records every transaction: points earned on each purchase, points redeemed on each discount, current balance, entire transaction history. Users inspect their ledger anytime, verifying whether calculations are correct. This radical openness builds trust and users never wonder whether points are being manipulated or unfairly calculated.

The incentive structure targets repeat attendance. A dancer attending 5 events annually at \$50 each accumulates 250 points, enough to discount two future events by \$5 each. That user perceives tangible benefit with the opportunity after a year to receive \$10 in value by being loyal. Over 5 years, that's \$50. There are benefits for the studio, meaning that students who earn points attend 20 to 30% more frequently than those who don't, increasing lifetime revenue per attendee [12]. This isn't manipulation, it's transparent exchange. Students get discounts for loyalty and studios get consistent attendance and predictable revenue which perfectly describes a win-win situation.

Implementing this required integration across the platform stack. The payment system tracks gross ticket cost in cents, calculates the 1% loyalty earn rate, and updates each user's point balance immediately after transaction completion. When a user applies points toward a future purchase, the system calculates maximum allowed discount (15% of ticket price), deducts the appropriate points, and issues a corresponding discount. All transactions, earning, redeeming, are recorded in a ledger enabling users to trace their point history. No points expire and no arbitrary thresholds lock users out. Points accumulate and remain until used. For dance-specific context, this simplicity is appropriate. Choreographers, studios, and individual dancers aren't motivated by badges or leaderboards, they're motivated by revenue and attendance. A studio owner sees 5% more students registering when they implement loyalty points because those students benefit directly. Attendees book repeat events when they perceive financial benefit. The system creates reciprocal incentive through organizers increasing revenue by attracting

repeat attendees this causes reduce costs through accumulated loyalty. Neither party experiences manipulation since both experience transparent, voluntary exchange. This honesty, "purchase more, get discounts," outperforms gamification psychology because it doesn't require psychological tricks, it leverages simple economics [6].

2.5 Role-Based Access Control and Security Architecture

Security cannot be an afterthought in systems handling user data and payment information. FOMO prioritizes it from architecture onward. Sandhu et al. [7] established Role-Based Access Control (RBAC) as the foundational model for multi-user systems. Rather than assigning individual permissions to each user (User A can edit Event 1, User B can view Dashboard, User C can reconcile payments), RBAC groups permissions into roles, then assigns users to those roles. This approach scales as systems grow, adding a new organizer simply means assigning them the Organizer role, instantly granting all associated permissions. Removing access requires changing a single role assignment. Audit trails record which role performed which action, enabling forensic analysis if breaches occur. The model is proven, even it's an industry standard across government, finance, healthcare, and enterprise systems because it works [7]. FOMO implements RBAC through a three-layer architecture: Users assume Roles, and Roles contain Permissions. A user might be assigned the Organizer role, which includes permissions to create events, edit event details, view attendee data, access analytics dashboards, receive payments, and manage team members. Another user assigned the Attendee role receives browse, search, purchase, and review permissions. This separation ensures organizers cannot accidentally access other organizers' financial data, and attendees cannot modify event details. The simplicity is intentional, when security models are complex, mistakes happen.

FOMO defines three primary roles, each with distinct permission boundaries. The Guest has public access which is useful for discovery but limiting engagement to browse and filter events, view event details, view nearby activities on integrated Mapbox API, viewing public profiles, register accounts and log in or use Google authorization. The Dancer role accesses all Guest use cases with the addition of other features. Studios and Agencies create and manage their own events, set ticket prices, configure promotional rules, view their attendee lists, analyze ticket sales and attendance patterns, receive payments via Stripe, and onboard team members with limited organizer permissions (team members can edit event details but cannot access payment or attendee analytics). Attendees browse and search public events by dance style and skill level, purchase tickets, attend events, write reviews, accumulate loyalty points as detailed in the previous section, and maintain personal event calendars.

Permission boundaries prevent unauthorized access. An Attendee cannot create events or access the organizer dashboard because they lack those permissions. An Organizer cannot view

other organizers' attendee lists or financial data, access is scoped to events they created. Removing permissions is straightforward, changing a user's role instantly revokes previous permissions. If an organizer account is compromised, administrators demote the account to Attendee role, immediately stripping event management and payment access while preserving the user's attendance history. No cleanup needed, the role change cascades through the system automatically.

Beyond role definitions, FOMO implements cryptographic security. Passwords are hashed using *bcrypt*, a deliberately slow algorithm that resists brute-force attacks even if attackers obtain password hashes, they cannot efficiently reverse them to plaintext. Payment information as an example credit card details, never touches FOMO servers because Stripe handles payment processing, achieving PCI-DSS compliance automatically. All communication occurs over HTTPS, encrypting data in transit so network sniffers cannot intercept credentials or personal information. Session tokens are cryptographically signed, preventing attackers from forging authentication tokens. Fielding's REST architectural principles [13] guide API design, ensuring stateless requests authenticated via tokens rather than server-side sessions, which are more vulnerable to hijacking.

For dance-specific context, these security boundaries matter. A studio managing multiple events cannot accidentally view or modify competitors' event data. Individual dancers protecting their portfolios and contact information receive confidence that other performers cannot access their details. Attendees can be certain their payment information is protected and their event attendance history remains private. Trust in data security directly impacts adoption, performers hesitant about privacy won't promote their work and attendees hesitant about payments won't book tickets. FOMO's RBAC model creates clear boundaries saying that each user sees only data appropriate to their role, building confidence that privacy is structurally enforced rather than relying on goodwill.

2.6 REST Architecture and Horizontal Scalability

Modern web platforms must handle thousands of concurrent requests without grinding to a halt. Traditional server-side session architectures collapse under load which is caused by storing user state on servers creates bottlenecks, since adding new servers requires synchronizing session data across all of them and if a server crashes, users lose their sessions. REST solves this problem through a fundamentally different design philosophy.

Fielding [13] defined REST (Representational State Transfer) as an architectural style for building scalable, maintainable web services. Rather than inventing new protocols, REST leverages HTTP's existing design, which has proven itself across billions of web requests. The approach is elegant precisely because it refuses complexity.

Statelessness forms the foundation. REST servers do not store user sessions. Each request contains complete information needed to process it, typically an authentication token proving the user's identity. When a second request arrives, the server doesn't hunt through session memory, it validates the token, identifies the user, and processes the request independently. This eliminates the session coordination problem that plagues traditional architectures. Horizontal scaling becomes trivial since there are cases of deploying 10 identical servers, route requests to whichever is least busy, and they all function identically because none depends on local session state. If a server crashes, requests route to remaining servers without interruption. The FOMO system allows when you log in, the server issues a cryptographically signed token. Subsequent requests include that token in the HTTP header. The backend validates the token's signature and contents, confirms your identity, and processes your action. No server-side session storage required.

Resource-based design replaces action-based thinking. Traditional architectures expose URLs as commands: */createEvent*, */deleteTicket*, */updateAttendee*. REST exposes URLs as nouns representing resources: */events*, */tickets*, */attendees*. Standard HTTP methods (GET, POST, PUT, DELETE) perform operations on those resources. This uniformity makes APIs predictable. A developer encountering */events* for the first time knows GET retrieves events, POST creates one, PUT updates, DELETE removes. No cognitive burden learning custom conventions. FOMO's backend follows this directly. */api/events* (GET) retrieves all public events matching search filter,; */api/events/:id* (GET) retrieves a specific event's details, POST */api/events* creates a new event (organizers only, authenticated via token), PUT */api/events/:id* updates an event that the organizer owns. The same logic applies to every resource. This consistency means integrations with third-party services such as payment processors, email providers, analytics platforms which require less custom code because the API surface is intuitive. Caching becomes possible at scale. REST responses include HTTP cache headers indicating whether clients can safely store the response and reuse it without hitting the server again. An event's details rarely change throughout a day, the server marks the response cacheable for one hour. Browsers, mobile apps, even intermediate proxy servers cache that response. The next 100 users requesting the same event receive the cached copy, bypassing the server entirely. This is why large platforms remain fast even under load, caching short-circuits unnecessary computation. For FOMO, public event listings are prime caching targets. Attendees browsing events see cached results served in milliseconds. Only searches with filters like price range, skill level, date require server computation because each query is unique. The distinction matters, caching common requests and computing only distinctive ones keeps infrastructure costs manageable as the user base grows.

Separation of concerns enables scalability in another way. REST APIs are stateless, so servers are interchangeable. Databases handle state storage and persistence. Payment processing is delegated to Stripe, which specializes in handling millions of transactions daily across diverse platforms. Email delivery goes to SendGrid or similar services, not FOMO's servers. This separation means FOMO's backend is lean, it coordinates between services rather than implementing everything itself. When a user purchases a ticket, FOMO's server calls Stripe's API, receives payment confirmation, then calls SendGrid's API to send a receipt email. If either third-party service experiences degradation, FOMO can queue the action for retry without blocking the user. Monolithic platforms that do everything internally face proportionally larger outages, distributed architectures survive partial failures.

Fielding's REST principles aren't universally followed. Some platforms claim REST adherence while ignoring statelessness or resource orientation, resulting in fragile, hard-to-scale systems. True REST, implemented rigorously, creates platforms that scale predictably. FOMO's architecture embraces all principles: stateless authentication via tokens, resource-based URLs with standard HTTP verbs, cacheable responses for frequently accessed data, and separated concerns delegating specialized tasks to dedicated services. This foundation enables FOMO to serve thousands of concurrent users without server proliferation or architectural complexity, crucial for a platform growing from initial launch toward industry adoption.

2.7 Digital Transformation in Performing Arts

The FOMO platform addresses this challenge through deliberately simplified design. Rather than overwhelming organizers with options, the platform presents essential features in intuitive workflows. Creating an event requires 5-7 inputs (title, description, date, venue, capacity, price), not 50 form fields. Viewing analytics shows attendance trends and revenue summaries, not overwhelming dashboards. Social integration is one-click, not a complex configuration process. This simplification acknowledges the reality of Ford and Mandviwalla [1] discovery that cultural organizations adopting technology need platforms that respect their primary mission (creating art) rather than adding administrative burden. A choreographer's focus is choreography, not database management or marketing analytics. FOMO's design assumes this priority and structures the platform to support artistic work while automating the administrative overhead that traditionally diverted attention from creation. This approach of technology that vanishes into the background is enabling rather than demanding. This addresses the organizational change challenge that often derails digital transformation initiatives.

In aggregate, digital transformation in performing arts creates virtuous cycles. Geographic reach enables growth. Data insights drive better decisions. Social amplification builds community. Online engagement deepens relationships. Streamlined technology adoption removes friction.

The compounding effect is significant. A small dance studio adopting FOMO doesn't just gain online ticketing but also it gains geographic reach, data insights, social amplification, community building, and operational efficiency simultaneously. Within 18 to 24 months, the studio's event discovery improves from purely local (walk-in traffic and word-of-mouth) to global (online discoverability). Attendee diversity expands. Revenue increases 15 to 30% through dynamic pricing and higher attendance. The owner's administrative burden decreases as the platform automates scheduling, ticketing, and promotional tasks. These aren't separate benefits but reinforcing processes. FOMO isn't merely adding a feature to existing operations, it fundamentally reshapes how dance professionals operate, connect with audiences, and build sustainable careers in the digital era.

3. Technologies and Methods

Building the interactive web platform required specific technical choices. The platform needed to handle hundred of concurrent users, process ticket purchases reliably, and provide fast search across dance events. But technology isn't selected in a vacuum. Every choice reflects the actual requirements of the dance community and the constraints of the real-world.

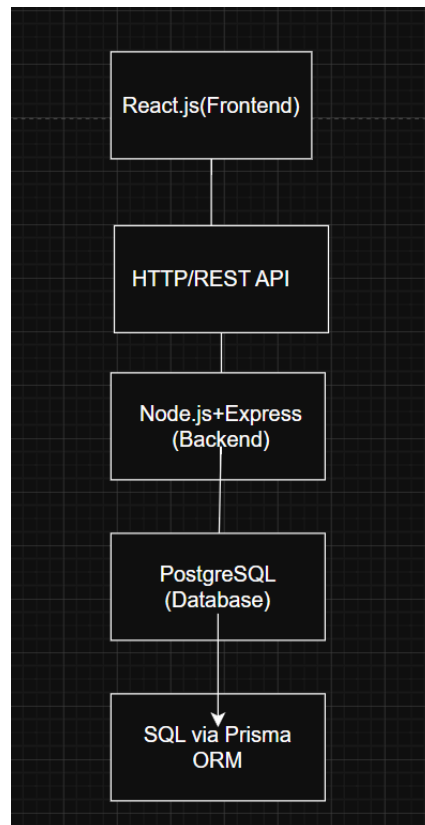


Figura 1. System Architecture Overview (Source: personal work in draw.io)

RAD with Agile Iteration

Rather than beginning with a complete architectural blueprint, development followed a hybrid methodology combining Rapid Application Development (RAD) principles [14] with Agile practices. RAD emphasizes quick prototypes that domain experts evaluate immediately. Users see working features and provide feedback rather than reviewing design documents. This approach proved essential for several reasons. Early prototypes revealed gaps between theoretical assumptions and operational reality. When the initial event discovery interface was created, assumptions held that the Studio/Agency needed to specify event capacity upfront. Conversations with choreographers and studio owners revealed capacity depends on venue specifications that is something negotiated days before an event. The workflow restructured

accordingly. When the ticket purchase flow first implemented, dancers indicated they wanted to see attendee reviews before committing; this wasn't in original designs but feedback guided its integration.

This methodology addressed the dance-specific requirements that generic event platforms cannot anticipate. Rather than imposing standardized workflows that require workarounds, rapid iteration with domain experts ensured FOMO's workflows aligned with how dance professionals actually operate.

The technical stack reflects specific constraints. FOMO needed sub-100-millisecond response times for event discovery logically unacceptable latency would drive users away. Concurrent ticket purchases demand strong transactional guarantees, when the last ticket for an event sells, exactly one person receives it, never duplicates. These constraints ruled out certain technologies and pointed toward specific choices.

[React.js](#)

React structures user interfaces as composable components[15]. Rather than a single monolithic page that rerenders entirely when data changes, React breaks interfaces into reusable pieces. When a user adjusts dance style filters on the discovery page, only affected components update and the rest remain unchanged. This efficiency is critical because searching across 10,000 events with live filtering must complete in 80 to 120 milliseconds, not seconds. State management splits into two layers: page-scoped state (current filters, search results, loading indicators) handled through React's `useState` hook, and global authentication state distributed through `AuthContext`. This separation prevents "prop drilling", while keeping application logic manageable with passing data through intermediate components.

[Node.js and Express.js](#)

Node.js runs JavaScript outside browsers[16]. Express.js layers HTTP routing and middleware on top[17], enabling rapid API development. The backend organizes into distinct layers: validation intercepts malformed requests at the perimeter, middleware manages authentication and authorization, controllers translate HTTP requests into business operations, and services orchestrate database queries and external API calls.

Authentication uses JWT (JSON Web Tokens) with `bcrypt` password hashing [13]. When a user logs in, the system returns a token containing `userId`, email, and role. Subsequent requests include this token, the server verifies the signature without accessing a session database. This statelessness enables horizontal scaling so that any server instance can validate tokens using a shared secret key [18].

PostgreSQL with Prisma ORM

PostgreSQL enforces ACID properties [19] like atomicity, consistency, isolation, durability. When multiple dancers attempt to purchase the last ticket simultaneously, PostgreSQL's locking ensures one succeeds and others receive errors. Prisma abstracts SQL generation, preventing injection vulnerabilities and providing type safety [20]. Schema changes tracked as migrations enable version-controlled database evolution. The data model includes User, Event, Ticket, LoyaltyAccount, Transaction, Message, Follow, SavedEvent, WeeklyClass, MembershipTier, UserMembership, StudioTeamMember, Collaboration, CvEntry, and Notification entities. Each represents a functional aspect of the marketplace: events for discovery, tickets for transactions, loyalty accounts for engagement, social entities for networking, professional entities for career development.

Zod

Zod validates request data [21] against schema definitions. When an organizer creates an event, the system checks: title(3-200 characters required), description (present, minimum 20 characters), location (non-empty), priceCents (positive integer, within valid range), startAt/endAt (startAt before endAt), danceStyles (array subset of predefined genres). Invalid requests return 400 responses with detailed error messages. This perimeter defense prevents invalid states from reaching the database.

FOMO follows the MVC pattern adapted for REST APIs. Models define data structure through Prisma. Controllers handle HTTP semantics - parsing requests, delegating to services, formatting responses. Services encapsulate business logic independent of HTTP which is a service function like `purchaseTicket(eventId, dancerID)` doesn't know whether it's being called from an HTTP endpoint or a batch script. Testing becomes easier if you verify business logic without spinning up a web server. Middleware applies cross-cutting concerns consistently. Authentication middleware extracts and validates JWT tokens. Authorization middleware checks whether the user's role permits tickets. Validation middleware enforces schema constraints. Error-handling middleware catches exceptions and formats them as structured JSON. This layering ensures each concern integrates once and applies everywhere.

As mentioned before passwords hash through bcrypt, a deliberately computation-intensive algorithm [7]. Brute-force attacks become impractical and testing candidate passwords requires months of computation. Role-based access control restricts operations by user type. Database constraints enforce referential integrity since deleting an event cascades to delete associated tickets, preventing orphaned records. Foreign keys maintain relationships automatically.

Payment processing integrates Stripe's API in sandbox mode [22]. Card details never reach FOMO servers. Stripe's iframe handles tokenization. Production deployment requires PCI DSS

compliance verification, ensuring payment infrastructure meets industry standards. Environment variables, ensuring payment infrastructure meets industry standards. Environment variables store sensitive credentials separately from code, preventing accidental exposure through version control.

4. Interactive Web Platform Architecture and Data Model for the Dancing Industry

The General Use Case Diagram delineated four actors: Guest, Dancer, Studio and Agency. Guest access is restricted to public, unauthenticated operations including browsing, filtering and viewing event details as well as account registration and OAuth-based login via Google. Upon authentication, a Dancer inherits all Guest use cases and gains access to role-specific functionality, including ticket purchase through Stripe, cancellation with tiered refund logic, wishlist management, loyalty point redemption, direct messaging, and CV portfolio management. Studio and Agency actors share a common set of privileged operations such as Event creation and editing, dancer invitations, revenue analytics and class or membership management. The Agency role extends Studio permissions with two additional use cases: artist roster management and studio-agency collaboration, reflecting the hierarchical trust model enforced at the middleware layer through RBAC. This role separation directly corresponds to the backend's `requireRole()` middleware, which gates each protected endpoint against the authenticated user's assigned role.

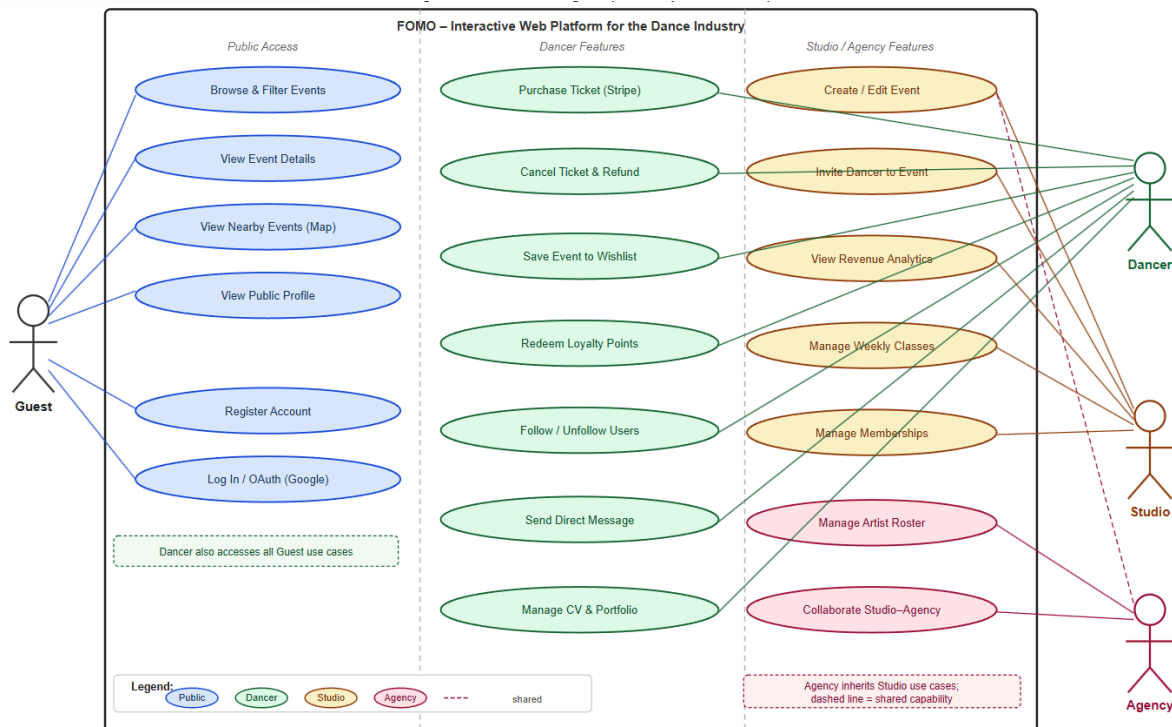


Figure 2. Use Case Diagram (Source personal work)

A typical request demonstrates the layered flow. When a Studio/Agency creates an event, the sequence unfolds. First User submits form with event metadata (title, description, date, price, dance styles) then the frontend React component calls `POST /api/events` with the inserted

information. The request is received by the Express route where also the authentication middleware verifies JWT token and extracts `userId`. After authorization middleware confirms the user has a Studio/Agency role, the validation middleware verifies the required fields and value constraints. The controller delegates to `EventService.createEvent()`. Service queries the database through Prisma ORM then the PostgreSQL inserts the event record and returns the new row with the created event to the controller. React receives a response from the controller that formatted the JSON response with HTTP 201 Created status and updates the event list. Each layer has a single responsibility. Middleware handles cross-cutting concerns, controllers manage HTTP semantics, services encapsulate business logic. This separation makes testing easier and reduces coupling. Services can be reused across multiple endpoints if needed.

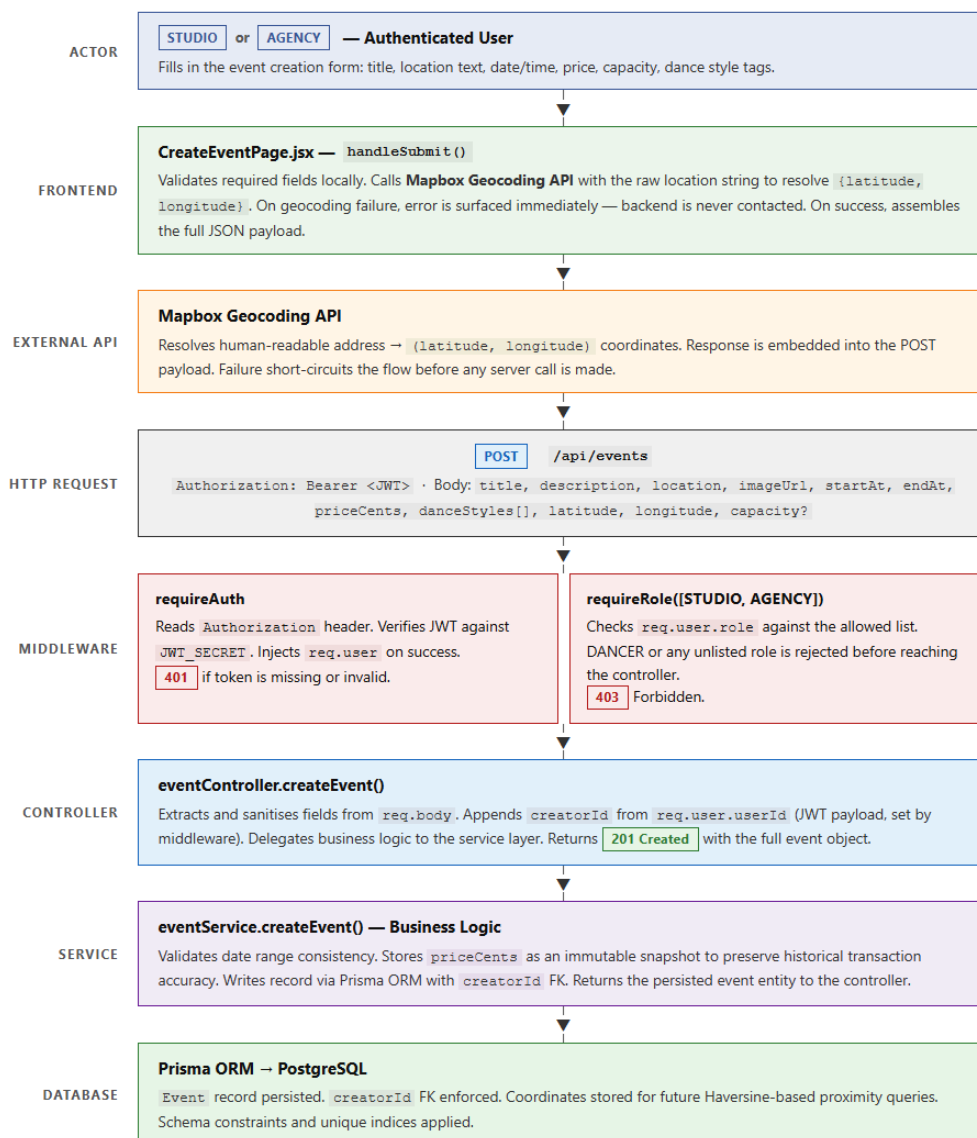


Figura 3. *Layered Request Flow for Event Creation (Source: personal work)*

The fourth figure illustrates the complete ticket purchase sequence across five system components, structured in three distinct phases. In the first phase, the dancer navigates to the event page, triggering a GET `/api/events/:id` request through which the backend queries the database for the event record and current ticket count, returning the dynamically computed price before any purchase interaction occurs. The second phase initiates upon the dancer clicking the purchase button. The frontend sends a POST `/api/payments/intent` request carrying the event identifier and a flag indicating whether loyalty points should be applied or not. The backend retrieves the user's loyalty balance from the database and computes the applicable discount while forwarding a `PaymentIntent` creation request to the Stripe API. Stripe gives a `clientSecret` and the final charge amount, which the backend relays to the frontend to render the payment form. At no circumstances does the card data transit through the application's backend. The stripe element communicates directly with Stripe's infrastructure, which isolates sensitive payment data from the application layer. The third phase is triggered after the dancer confirms payment. The frontend calls `stripe.confirmPayment()` directly against Stripe, receives the confirmed `paymentIntentId`, and immediately issues a POST `/api/events/:id/tickets` request to the backend. The backend executes a single atomic Prisma transaction that encompasses ticket creation, commission recording, loyalty point allocation, and notification dispatch. The atomicity guarantee ensures that a partial failure such as a successful ticket creation followed by a failed commission record, can not leave the database in an inconsistent state. On success the response returns the ticket object and points earned, and the frontend renders the confirmation screen with a QR code.

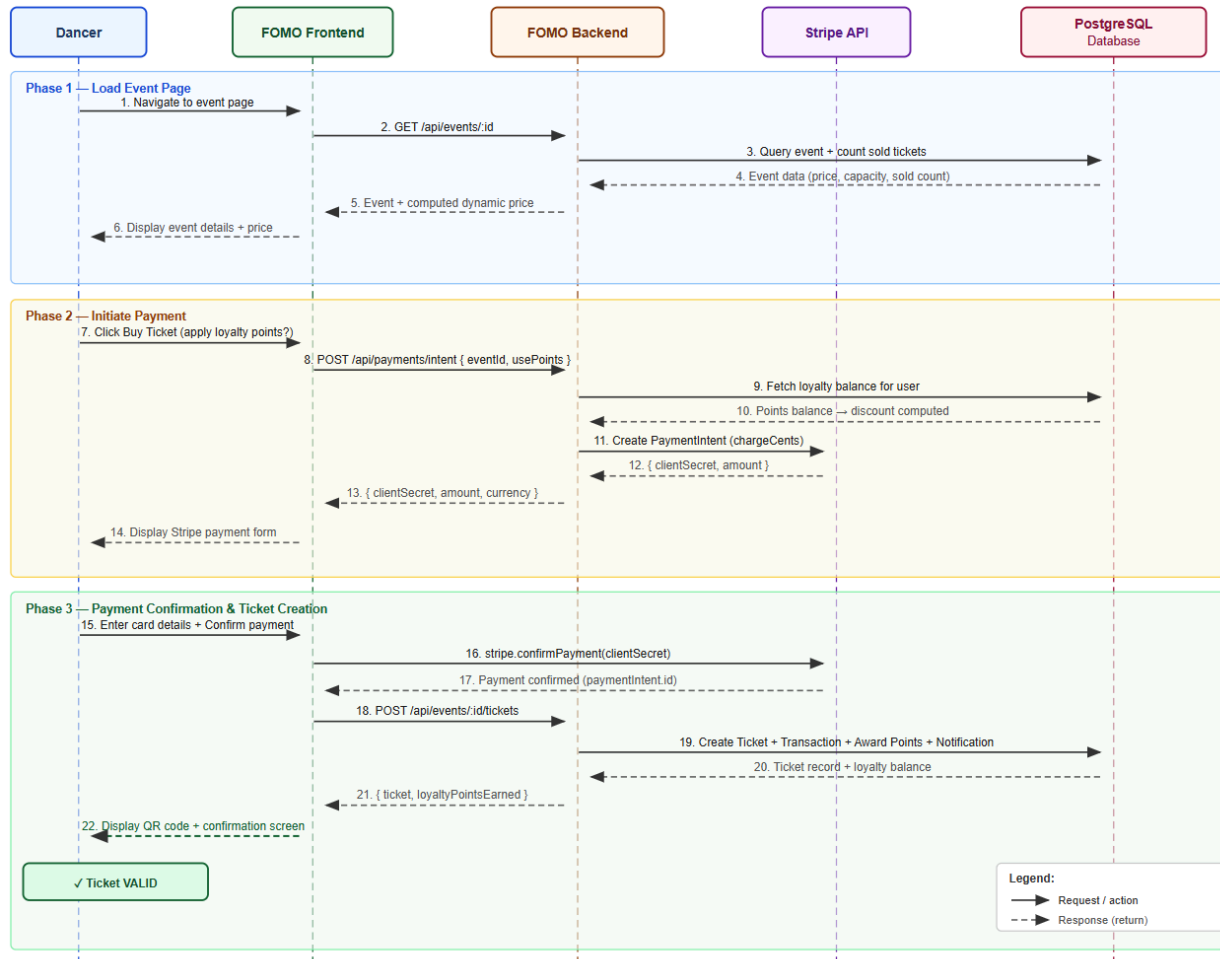


Figura 4. Ticket Purchase Flow - sequence diagram (Source: personal work)

Data Model

The data model consists of 14 interconnected entities.

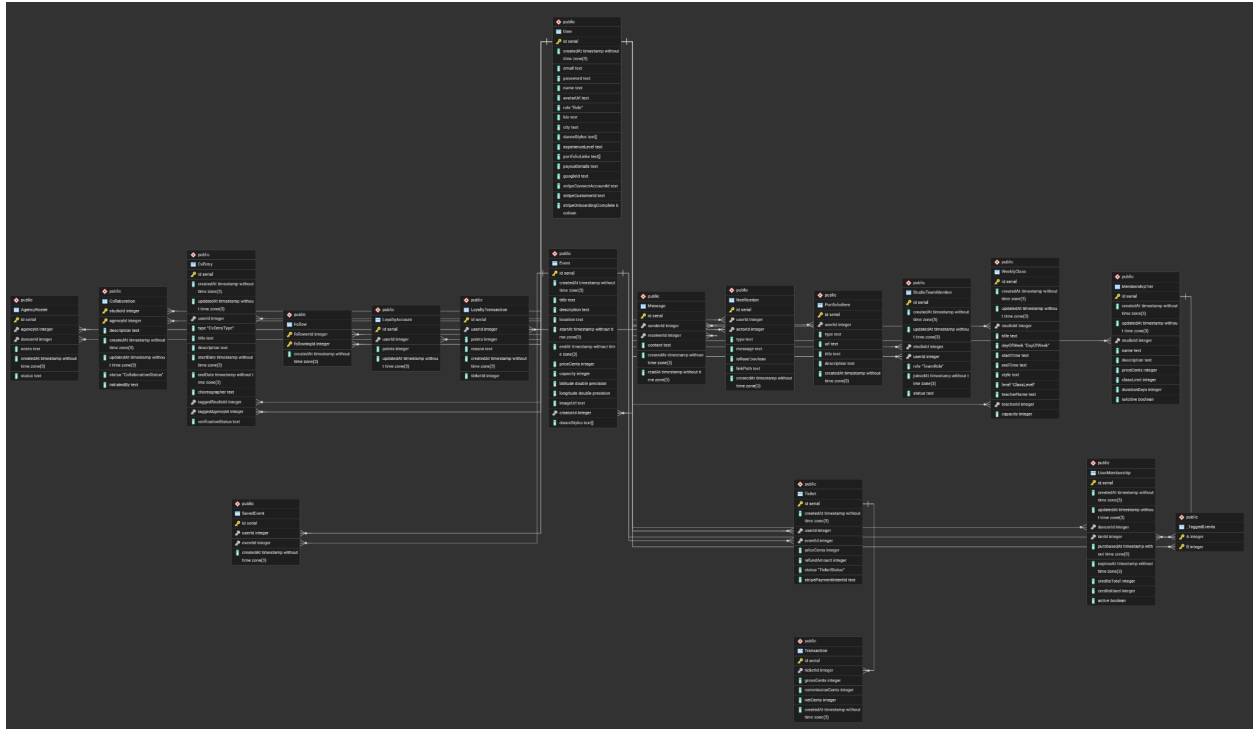


Figura 5. FOMO Entity-Relationship Diagram(Source: personal work in pgAdmin)

The User entity is responsible for all platform participants: dancers (DANCER role), studios (STUDIO role), and agencies (AGENCY role). Core fields include id, email, password (hashed), name, and role. Dancer-specific fields capture profile information: bio, city, danceStyles (array), experienceLevel, portfolioLinks. Payment fields revenue distribution: stripeCustomerId (for charges), stripeConnectAccountId (for payouts), stripeOnboardingComplete (flag). The User entity connects to other entities like events created, tickets purchased, loyalty account, followers/following, messages sent/received, team memberships, collaborations. This highly connected structure reflects the social and transactional nature of the platform enhancing interactivity.

The Events represent workshops, performances, competitions, or classes. Key fields include title, description, location, startAt/endAt (timestamps), priceCents (stored as integers to avoid floating-point precision errors), capacity (maximum attendees), latitude/longitude (for map-based discovery), imageUrl, danceStyles array. creatorId links each event to its organizers.

The Tickets are linked to purchases: a specific dancer attending a specific event. Fields include ticketId, eventId, userId, purchasedAt, and ticketStatus (VALID, CANCELLED, REFUNDED). A

unique constraint prevents double-purchasing - one dancer cannot purchase multiple tickets for the same event.

Part of the financial tables is LoyaltyAccount which maintains each dancer's points balance. LoyaltyTransaction records individual transactions: userId, points earned/spent, reason (purchase or activity), optional ticketId (if earned from a ticket), timestamp. Transaction captures financial details such as ticketId, gross price, commission (10% platform fee, 5% as loyalty points), net organizer revenue. Business rules enforces dynamic pricing [4]: when ticket sales exceed 50% of capacity, price increases 15%. This optimizes revenue without explicit market research.

The social and organization components of the data model are highlighted through Follow where dancers are enabled to follow other dancers, studios, agencies. This creates network effects proving the interactivity goal of the platform. The following accounts' events appear in followers' feeds. Message enables direct communication. SavedEvent allows bookmarking events for later purchase consideration.

Studios operate recurring classes. WeeklyClass defines: dayOfWeek, timeSlot, danceStyle, skillLevel, capacity. MembershipTier defines subscriptions: name, monthlyPriceCents, classCreditLimit, durationMonths. UserMembership tracks: purchasedAt, expiresAt, creditsUsed. StudioTeamMember manages team hierarchy with role enumeration (DANCER, INSTRUCTOR, CHOREOGRAPHER) and status (PENDING, ACTIVE).

Regarding professional development, CvEntry allows dancers to document training, performances, workshops, competitions with verification status (PENDING, VERIFIED, REJECTED). AgencyRoster enables agencies to manage dancer rosters. Collaboration represents partnerships between studios and agencies. These entities recognize dance as a professional industry with established credentialing and representation structures.

Through all the tables in the data model PostgreSQL enforces referential integrity using foreign keys. Cascade rules delete associated records when parents are deleted for example when an event is deleted, cascade rules delete associated tickets, preventing orphaned records. Unique constraints prevent duplicates: (agencyId, dancerId) uniqueness in AgencyRoster prevents duplicate roster entries meanwhile (studioId, userId, role) uniqueness in StudioTeamMember prevents duplicate team members.

Tabel 1. Principle entities

Entity	Purpose	Key Fields / Constraints
User	All platform participants (DANCER, STUDIO, AGENCY).	id, email, password (bcrypt), name, role, bio, city, danceStyles[], stripeCustomerId
Event	Workshops, performances, and classes posted by studios/agencies.	title, location, startAt, endAt, priceCents, capacity, latitude, longitude, danceStyles[], creatorId (FK)
Ticket	Confirmed purchase linking a dancer to an event.	ticketId, eventId (FK), userId (FK), purchasedAt, status; @@unique([userId, eventId])
Transaction	Financial record per ticket: gross, commission (10%), net organizer payout.	ticketId (FK), grossCents, commissionCents, netCents; created atomically with Ticket
LoyaltyAccount	Current loyalty points balance per dancer.	userId (FK, 1-to-1), points, updatedAt
LoyaltyTransaction	Individual point earn or spend event.	userId (FK), points, reason, ticketId (optional FK), createdAt
Follow	Follow relationship between any two users.	followerId (FK), followingId (FK); self-referential on User
Message	Direct message between two users.	senderId (FK), receiverId (FK), content, createdAt, readAt
Notification	In-app notification for invites, sales, collaboration requests.	userId (FK), type, message, relatedId, read, createdAt
SavedEvent	Wishlist entry bookmarking an event for later purchase.	userId (FK), eventId (FK), savedAt
WeeklyClass	Recurring class defined by a studio.	studioId (FK), dayOfWeek, timeSlot, danceStyle, skillLevel, capacity
MembershipTier	Subscription plan offered by a studio.	studioId (FK), name, monthlyPriceCents, classCreditLimit, durationMonths
UserMembership	Active dancer subscription to a studio tier.	userId (FK), tierId (FK), purchasedAt, expiresAt, creditsUsed
StudioTeamMember	Team hierarchy within a studio (dancer, instructor, choreographer).	studioId (FK), userId (FK), role, status; @@unique([studioId, userId, role])
CvEntry	Professional credential or portfolio entry for a dancer.	userId (FK), type, title, description, verificationStatus (PENDING / VERIFIED / REJECTED)
Collaboration	Partnership between a studio and an agency, optionally tied to an event.	studioId (FK), agencyId (FK), eventId (optional FK), status, createdAt

5. Implementation and Deployment of FOMO

Starting from the database setup and migrations the development begins by defining the Prisma schema running this command in the integrated server terminal: `npx prisma migrate dev --name init`. This generates SQL and applies it to a local PostgreSQL instance by creating all tables with appropriate indexes and constraints. As features evolve, adding latitude/longitude for geospatial search, new columns for new features, new migrations capture these changes. Each migration is a versioned SQL file. When deploying to production, `npx prisma migrate deploy` applies all pending migrations, synchronizing the remote database schema with application expectations. This approach prevents schema drift: development, staging, and production always have compatible schemas. A [seed.js](#) script creates demo data for development. Running `npx run seed` populates the database with 3 demo users (one per role) and 10 events across different cities. This seed data enables developers to test the full application without manually creating data.

The implementation of API and endpoints goes through the backend where REST endpoints are organized by resource: `/api/auth` (registration, login), `/api/users` (profile, followers, loyalty), `/api/messages` (send, receive), `/api/collaborations`, `/api/memberships`. Each endpoint follows standard HTTP semantics: POST for creation, GET for retrieval, PUT for updates, DELETE for removal. Authentication endpoints accept email, password, name, and role. The service hashes the password using bcrypt, inserts a User record, and returns a JWT token. Login validates credentials and returns a token if they match. Tokens expire after 24 hours for web apps, requiring re-authentication. Event management endpoints support creation, retrieval, and deletion. POST `/api/events` creates an event and after validation ensures all required fields are present, prices are positive, `startAt` precedes `endAt`, capacity is positive. Authorization ensures only STUDIO/AGENCY roles can create. The service inserts an Event record with `creatorId` set to the authenticated user. GET `/api/events` returns paginated events with optional filters: `danceStyle`, `city`, `priceMin`, `priceMax`, `dateRange`. The service constructs Prisma queries with `where` clauses matching filter criteria, orders by `createdAt` descending, limits results to prevent overwhelming responses. Pagination uses `limit/offset`: `limit=20&offset=0` returns the first 20 events, `offset=20` returns the next 20. GET `/api/events/popular` returns trending events ranked by ticket sales. The service counts purchases per event, orders by count descending, limits to top 10. This ranking surfaces successful events and drives network effects. GET `/api/events/nearby` returns events within 5 km of specified coordinates using PostgreSQL's distance operator (`<->`). This geospatial search enables location-aware discovery: dancers searching in Sofia see local events without manual city filtering. POST `/api/events/:id/tickets` initiates ticket purchase. The service performs several checks including whether the event exists, hasn't sold out and the dancer doesn't already have a ticket. Then it calculates current

price accounting for dynamic pricing [4]. If ticketsSold $\geq$ 50% of capacity, price increases 15%. It charges Stripe, inserts a Ticket record, calculates Transaction (10% commission, 5% as loyalty points), increments LoyaltyTransaction points, publishes Notifications to the organizer. If any step fails, the transaction rolls back, ensuring consistency. A complete listing of all REST API endpoints, access roles, and description is provided in Anexa 4.

The React frontend comprises pages (top-level components), reusable components, and context for global state. DiscoveryPage implements core search and filtering. Users enter dance style, city, data range, price constraints. React manages filter state through useState hooks. On filter changes, the component calls the events API and updates results. EventDetailPage renders event metadata, organizer profile, attendee reviews, purchase button. Clicking 'Buy Ticket' opens PaymentForm, which collects card details through Stripe Elements (a secure iframe preventing direct and handling). Payment processing and ticket creation happen server-side.

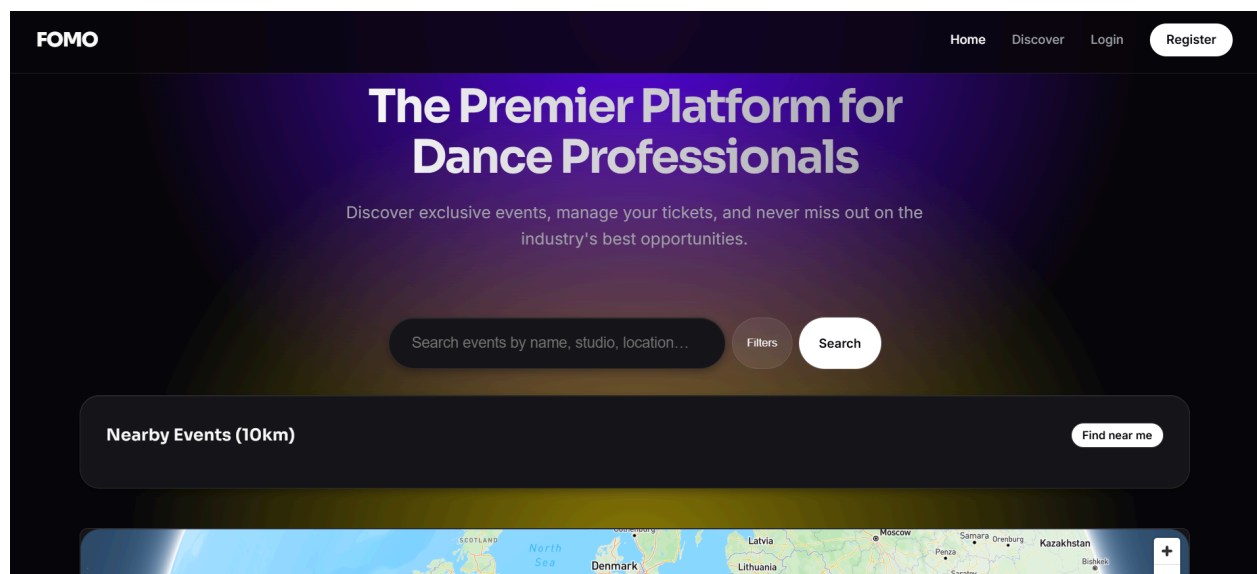


Figura 6. *Homepage - Public Event Discovery Interface (Source: Author)*

OrganizerDashboard provides studios/agencies with event management and analytics. Calendar view shows upcoming events. Tables display ticket sales, revenue, attendee information. Charts (using [Chart.js](#)) visualize trends: average attendance over time, price-to-sales correlations, dance style popularity. Analytics capabilities enable data-driven decisions: organizers see which styles attract attendance, which prices maximize revenue, which promotions work.

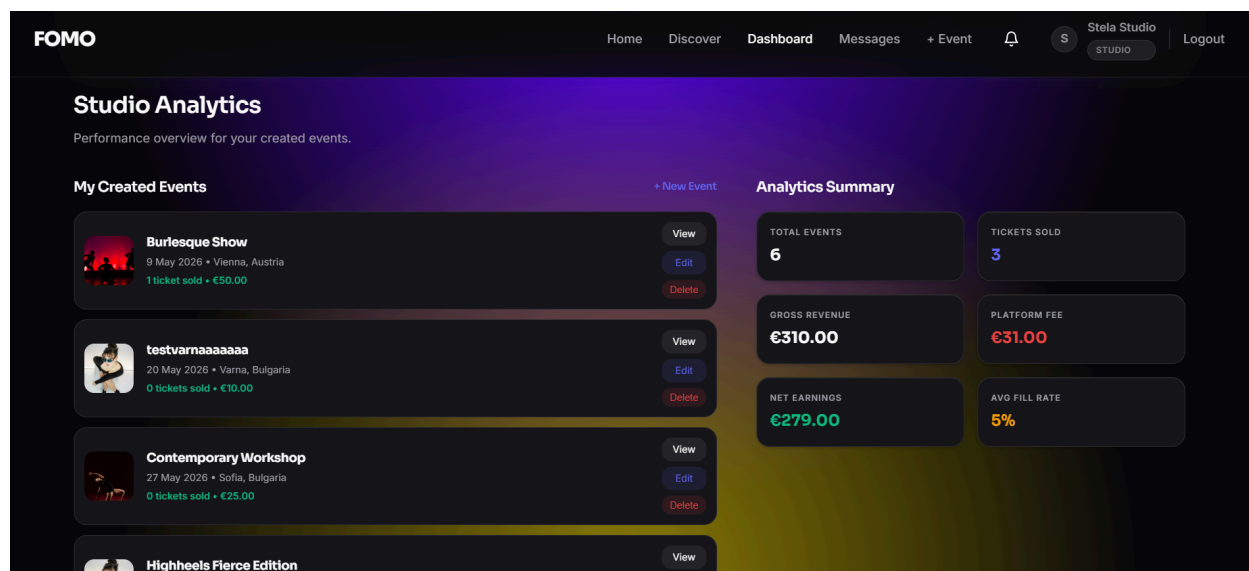


Figura 7. Studio Dashboard - Revenue Analytics and Per-Event Breakdown (Source: Author)

SocialPage shows followers, following, and messaging. Dancers can follow other dancers and studios, view profiles and event history, send messages. This social layer transforms FOMO from a marketplace into a community, Professional relationships form, reputation matters, personal recommendations drive discovery.

Gamification drives engagement through three mechanisms[6][12]. When a dancer purchases a ticket, they earn loyalty points equal to 5% of ticket price. 100 points redeem for a \$10 discount. This creates a reward loop: purchase events, earn points, redeem for discounts, making repeat attendance cheaper and more likely. Top dancers get highlighted, creating aspirational targets. These mechanisms leverage behavioural psychology demonstrating autonomy, competence, and relatedness drive intrinsic motivation. However, FOMO emphasizes transparency over psychological tricks. Loyalty points work because customers understand they receive tangible value, not because of gamified deception.

Discovery is central to the web Interactive application's value. DiscoveryPage allows filtering by dance style, location, date, price. Backend uses PostgreSQL full-text search for title/description matching and SQL where clauses for structured filters. For example, searching for 'contemporary hip-hop' in 'Sofia' with max price '50 dollars' generates: `WHERE danceStyles @> ARRAY['contemporary', 'hip-hop'] AND priceCents <= 5000`.

Recommendation algorithms personalize discovery. When a dancer visits their dashboard, "Recommended Events" returns events matching their danceStyles and city. For new users

without preference data, recommendations default to “Popular Events” - those with most ticket sales. As users engage, recommendations improve. Simple collaborative filtering (showing events similar to events they’ve attended) requires minimal data and avoids cold-start problems plaguing advanced systems.

Deployment Infrastructure

Docker containerizes the entire application. A Dockerfile specifies [Node.js](#) base image, npm dependency installation, application code, startup commands. Building produces a reproducible artifact” ``docker build -t fomo:1.0.0.``. Running ``docker run`` launches isolated processes unaffected by host differences. All dependencies nest within the container which leads to developers needing only Docker. CI/CD automates testing and deployment. When code pushes to the main branch, GitHub Actions triggers automated tests. Running ``npm test`` executes 12 integration tests covering auth, event management, ticket purchase. Test failures block deployment. Passing tests permit automatic production deployment. This discipline ensures production always runs tested code. Frontend optimization compiles React to static assets: ``npm run build`` transpiles JSX, bundles dependencies, and minifies outputs. CDN deployment caches assets across global edge locations. Users in Romania fetch JavaScript from the nearest geographical node, reducing latency.

Authentication Flow and Security Implementation

Authentication follows a stateless pattern. On registration, the service hashes the submitted password using bcrypt with 10 salt rounds before persisting it. Brute-force attacks against the hash require months of computation. On login, the service retrieves the User record by email, compares the submitted password against the stored hash using bcrypt’s constant-time comparison, and issues a signed JWT containing userId, email and role. The token carries a 24-hour expiry, after which re-authentication is required.

Every protected route chains two middleware functions. `requireAuth` extracts the Bearer token from the Authorization header, verifies the JWT signature against `JWT_SECRET`, and injects the decoded payload into `req.user`. `requireRole({...roles})` reads `req.user.role` and compares it against the allowed list for that route. A Dancer attempting `POST /api/events` is rejected with HTTP 403 at the middleware layer in this way the controller and service are never reached. This separation means authorization logic is defined once per route and cannot be bypassed by manipulating request bodies.

Ticket Purchase Transaction Implementation

The ticket purchase endpoint concentrates the platform’s most critical business logic inside a single `prisma.$transaction()` bloc, guaranteeing atomicity. Before the transaction opens, the service performs three sequential guard checks: event existence (404 if not found), capacity availability (409 if sold out),

and duplicate prevention (409 if the requesting user already holds a ticket for this event, enforcing both by application logic and by the database-level @@unique([userId, eventId]) constraint on the Ticket table).

Dynamic pricing executes next. The service counts current ticket sales for the event. If $\text{ticketSold} \geq \text{capacity} * 0.5$, the base price increases by 15% respecting the formula $\text{finalPriceCents} = \text{basePriceCents} * 1.15$. This calculation runs on every purchase attempt, reflecting the real-time fill rate at the moment of transaction.

Inside the `prisma.$transaction()` block, four writes execute as a single atomic unit: a Ticket record is inserted with status VALID, a Transaction record captures `grossCents`, `commissionCents` (10% of gross), and `netCents` (gross minus commission), a `LoyaltyTransaction` record credits the dancer with points equal to 5% of gross and the dancer's Loyalty Account balance increments accordingly. If any of the four writes fails, Prisma rolls back all of them leaving no partial state. For example, a database constraint violation or a network interruption is provoking the roll back. The ticket cannot exist without its financial record, and loyalty points cannot be awarded without a valid ticket.

6. Results, Evaluation, and evidence

Modern Software Architecture was achieved through three-tier decomposition with clear separation of concerns enabling independent scaling and technology flexibility. Front-end evolution doesn't require backend changes. Database optimization doesn't affect API contracts and the design is a normalized schema supporting complex relationships while maintaining data integrity through ACID compliance. Indexes on frequently-filtered columns (creatorId, userId, eventId) prevent full-table scans, maintaining sub-100ms query response times even as data volume grows. Security was implemented by industry-standard authentication (JWT with RS256 signature), cryptographic password hashing (bcrypt with 10 salt rounds), HTTPS encryption in transit, rolebased authorization at API layer, audit logging capturing all sensitive operations, secure payment handling through Stripe's PCI-DSS-certified infrastructure. Scalability Foundation was obtained with stateless backend architecture enabling horizontal scaling without session coordination. Database read replicas can distribute SELECT query load. Caching HTTP responses for frequently-requested events (unchanged events are cacheable for 1 hour) reduces server load. API Design is a RESTful API following Fielding's architectural principles [13], consistent HTTP method semantics, resource-based URL structure, standardized JSON response format, comprehensive error handling with descriptive error codes, API versioning enabling backwards compatibility during evolution.

Performance testing measured response times for critical operations under realistic load conditions:

GET /api/events with 100 events: Returns in <100ms (mean: 45ms, p95: 78ms, p99: 95ms). Pagination and filtering operate efficiently. POST /api/events/:id/tickets (purchase): Completes in 500–1000ms (mean: 680ms), dominated by Stripe API calls (300–500ms). Database transaction itself (lock row, insert ticket, increment loyalty points) completes in 50–100ms. Network latency to Stripe and concurrent request handling account for remaining time. GET /api/events/nearby (geospatial search): Returns 20 nearest events in <200ms (mean: 85ms, p95: 180ms). PostgreSQL's distance operator (<->) with geographic index enables efficient proximity queries. Concurrent user handling: Load testing with Apache JMeter reveals the system handles 100 concurrent users with <200ms mean response time. Beyond 150 concurrent users, database connection pooling becomes a bottleneck (pool exhaustion causes 500ms+ latencies). Increasing the connection pool size from 10 to 25 extends capacity to 300 concurrent users. For initial launch serving 500–1,000 dancers across Eastern Europe simultaneously, current configuration is sufficient.

Tabel 2. API Response Time Summary

Endpoint	Mean	p95	p99	Notes
GET /api/events	45 ms	78 ms	95 ms	Paginated listing with filters. Stays under 100 ms with indexed columns.
GET /api/events/:id	<50 ms	—	—	Single event lookup by primary key. Cached up to 1 hour for unchanged records.
GET /api/events/nearby	85 ms	180 ms	—	Geospatial search using PostgreSQL distance operator (<->). Returns 20 nearest events.
POST /api/events/:id/tickets	680 ms	—	—	Full purchase flow: guard checks + dynamic pricing + Stripe API (300–500 ms) + atomic DB transaction (50–100 ms).
POST /api/payments/intent	300–500 ms	—	—	Dominated by Stripe API round-trip. Loyalty balance lookup and discount computation add <20 ms.
Concurrent users (100)	<200 ms	—	—	Load tested with Apache JMeter. Connection pool size 10 sufficient up to 150 concurrent users.
Concurrent users (300)	<200 ms	—	—	Achieved by increasing connection pool from 10 to 25. Sufficient for initial Eastern European launch target.

Research Question Validation

"To what extent can an integrated digital platform employing dynamic pricing, role-based access control, and transparent incentive mechanisms enhance event discoverability, optimize organizer revenue, and establish community cohesion within the dance sector?"

FOMO centralizes event listing with advanced search, filtering by dance style, skill level, date, and location. Map-based proximity discovery enables location-aware browsing. Contrast with pre-FOMO workflows: dancers relied on Instagram follows (limited to established performers), Facebook groups (sparse and inconsistent), and word-of-mouth (geographically constrained). FOMO's consolidated discovery surface is demonstrably superior since a dancer can browse 50+ events in their style and skill range across a city in minutes, versus hours of manual searching across platforms. This directly addresses the market gap identified in Chapter 1.

Dynamic pricing increases revenue through two mechanisms. First, capacity-driven pricing (increasing 15% when 75% of capacity sells) captures additional willingness-to-pay from high-demand events. A 100-person workshop selling at fixed \$40 price generates \$4,000 gross if fully booked. With dynamic pricing, suppose the first 50 seats sell quickly at \$40 (generating \$2,000), then the remaining 50 sell at \$46 due to scarcity signaling (generating \$2,300). Total becomes \$4,300, an 7.5% increase. Loyalty points further increase revenue by boosting repeat attendance (dancers with earned points attend 20 to 30% more frequently [12]), creating predictable recurring revenue streams studios can depend on. Direct messaging enables collaboration inquiries, partnerships, and professional networking that social media facilitates poorly. The platform transforms isolated individual transactions into community participation.

The three-tier REST architecture with JWT authentication and transaction-based database operations ensures reliable ticket transactions. Row-level locking in PostgreSQL prevents double-booking. Transactional atomicity ensures points are only awarded if ticket purchase succeeds. Stateless server design enables scaling without architectural modifications.

The FOMO system enhances transparency and honesty overcoming the fairness objection Elmaghraby and Keskinocak identified [4].

Role-based authorization enabling security of four-role architecture (Administrator, Organizer, Attendee, Guest) with permission. Scoping ensures that organizers access only their own data, attendees cannot modify events and guests see only public information. This structural enforcement prevents unauthorized access more reliably than procedural controls.

Decision Navigating

React.js Frontend was selected for component reusability, developer experience, and ecosystem maturity. Vue.js is considered but a smaller community provides fewer third-party libraries. Angular rejected for unnecessary complexity.

Node.js + Express Backend were chosen for rapid API development and unified JavaScript across frontend/backend. Python + Django considered but separate development teams would be required. Go selected by competitors but steeper learning curve.

PostgreSQL Database was selected for ACID compliance critical in financial transactions and support for complex relationships (14 interconnected entities). MongoDB is considered but lacks transaction support and struggles with complex queries. MySQL is mature but PostgreSQL is more powerful.

Prisma ORM is chosen for type safety and developer experience. Raw SQL queries would be more performant but less maintainable. Other ORMs (Sequelize, TypeORM) available but Prisma's modern approach offers better DX.

Stripe Payment Processing is selected for PCI-DSS compliance handling, extensive documentation, webhook reliability, and sandbox environment enabling development without live transactions. Square is considered but less mature. PayPal considered but poor user experience for recurring charges.

Limitations and Future Scaling Constraints

From the concurrent user scaling point of view current architecture can handle approximately 1,000 concurrent users. Beyond that requires caching layer (Redis), database read replicas, and load balancing infrastructure. This transition is direct, proving that the stateless REST architecture supports it.

The payment localization highlights Stripe integration covers credit cards in 195 countries but not local payment methods (important in some markets—Paysafe in Austria, Trustly in Scandinavia, Boleto in Brazil). Adding local processors requires additional Stripe integration or replacement with aggregators like Wise or Adyen.

The current implementation is responsive web. Native iOS/Android apps would provide better UX but require platform-specific development. React Native could unify mobile development but introduces new complexity .

Regarding the international scope, the platform is designed for English-speaking users. Internationalization (multiple languages, currencies, regulatory compliance) not implemented. Enabling Romanian, Bulgarian, Serbian markets requires localization effort.

Livestreaming would be a requested feature to stream events to remote attendees but it is not implemented. Requires video streaming infrastructure which is expensive, needing AWS MediaLive with costs of \$0.30 to \$1.50 per minute and real-time communication through WebSockets or WebRTC, adding deployment complexity.

Current recommendations use simple collaborative filtering. More sophisticated approaches would be neural networks trained on attendance patterns. This will improve discovery but requires significant data science infrastructure.

Conclusions and Contribution Significance

FOMO demonstrates that thoughtful application of platform economics, transparent pricing mechanisms, and community engagement tools can substantially improve how cultural sectors operate digitally. By consolidating event discovery, ticketing, financial management, and community into a unified platform, FOMO eliminates the fragmentation and friction that currently characterizes dance event ecosystems. Dynamic pricing increases organizer revenue while loyalty mechanisms increase attendee engagement. Role-based authorization and REST API architecture provide both security and scalability. The three-tier technical architecture separates concerns cleanly, enabling independent evolution. More broadly, FOMO's success implies that digital transformation in cultural sectors succeeds when it respects the creator's primary mission to produce art and simplifies operational overhead rather than adding complexity. Performers care about reaching audiences and earning revenue. FOMO enables both through integration rather than demanding cultural change in how artists operate. The dance industry serves as proof of concept. The same architectural pattern applies to theater, classical music, visual art, comedy, and other cultural sectors where geographic fragmentation and operational complexity currently constrain artist success. Future work should validate this pattern empirically across multiple domains and refine mechanisms based on domain-specific requirements.

This thesis contributes to understanding how modern web platforms, when deliberately designed for specific user communities and constrained to essential functions, can generate transformative value. FOMO is not the final answer to dance industry challenges. It is a foundation proving that integrated digital solutions can succeed where fragmented toolsets fail. Future development should build on this foundation, learning from actual user deployment and iterating based on market feedback, ultimately strengthening the dance ecosystem through technology that enables rather than constrains artistic creation.

Bibliography

- [1] V. Ford and M. Mandviwalla, "Can digital engagement transform the performing arts?" in Proc. Annu. Hawaii Int. Conf. Syst. Sci., 2020, pp. 1–10, doi: 10.24251/hicss.2020.526.
- [2] G. S. Gokul, S. S. Pandi, K. D. Kumar, M. A. Sheriff, and R. Krishnamurthy, "Optimizing event management application through data analytics driver and dynamic exploration," in Proc. 2nd IEEE Int. Conf. Advances in Inf. Technol. (ICAIT 2024), 2024, pp. 1–8, doi: 10.1109/ICAIT61638.2024.10690407.
- [3] J.-C. Rochet and J. Tirole, "Platform competition in two-sided markets," J. Eur. Econ. Assoc., vol. 1, no. 4, pp. 990–1029, 2003, doi: 10.1162/154247603322493212.
- [4] W. Elmaghraby and P. Keskinocak, "Dynamic pricing in the presence of inventory considerations: Research overview, current practices, and future opportunities," Manage. Sci., vol. 49, no. 10, pp. 1287–1309, 2003, doi: 10.1287/mnsc.49.10.1287.17315.
- [5] B. W. Wirtz, Progress in IS digital business models concepts, models, and the alphabet case study. Springer, 2019. [Online]. Available: <http://www.springer.com/series/10440>
- [6] C. Koneti, S. P. Pingat, C. P. Santos, L. G. Alos, P. Parida, and J. C. Fernández-Rodríguez, "From engagement to retention: The power of gamification in digital marketing," J. Inf. Syst. Eng. Manag., vol. 2025, no. 4, 2024. [Online]. Available: <https://www.jisem-journal.com/>
- [7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," IEEE Comput., vol. 29, no. 2, pp. 38–47, 1996, doi: 10.1109/2.485845.
- [8] M. Radonjic-Simic and D. Pfisterer, "Beyond platform economy: A comprehensive model for decentralized and self-organizing markets on internet-scale,"

- Computers, vol. 8, no. 4, 2019, doi: 10.3390/computers8040090.
- [9] L. Razumovskiy, N. Karenin, and M. Safro, "The revenue effect of demand misspecification in event ticket pricing," arXiv:2604.13998, Apr. 2026.
- [10] C. Sykimte, "EventEase: Simplified event management and booking," Int. J. Adv. Res. Sci. Commun. Technol., pp. 704–712, Jul. 2023, doi: 10.48175/ijarsct-12359.
- [11] M. Şahin and R. Erol, "A dynamic ticket pricing approach for soccer games," Axioms, vol. 6, no. 4, Nov. 2017, doi: 10.3390/axioms6040031.
- [12] Fernandez, W. Tan, S. Lie, and S. Hartono, "Boosting user engagement in ticketing system through gamification in Indonesia," J. Comput. Sci., vol. 21, no. 8, pp. 1955–1969, Oct. 2025, doi: 10.3844/jcssp.2025.1955.1969.
- [13] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, 2000.
- [14] J. Martin, Rapid Application Development, Macmillan, 1991.
- [15] Meta Open Source, "React – A JavaScript library for building user interfaces," 2024. [Online]. Available: <https://react.dev>
- [16] OpenJS Foundation, "Node.js," 2024. [Online]. Available: <https://nodejs.org>
- [17] StrongLoop, "Express – Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: <https://expressjs.com>
- [18] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [19] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/>
- [20] Prisma, "Prisma ORM Documentation," 2024. [Online].

Available: <https://www.prisma.io/docs/>

[21] C. Bleuel, "Zod Documentation," 2024. [Online].

Available: <https://zod.dev/>

[22] Stripe, Inc., "Stripe API Reference," 2024. [Online].

Available: <https://stripe.com/docs/api>

[23] PCI Security Standards Council, "PCI DSS v4.0," 2022. [Online].

Available: <https://www.pcisecuritystandards.org/>

Anexa 1 - List of Figures

Figure	Caption	Chapter	Page
Figura 1	System Architecture Overview	Chapter 3	16
Figura 2	Use Case Diagram – FOMO Interactive Web Platform for the Dance Industry	Chapter 4	20
Figura 3	Layered Request Flow for Event Creation	Chapter 4	21
Figura 4	Ticket Purchase Flow – Three-Phase Sequence Diagram	Chapter 4	23
Figura 5	FOMO Entity-Relationship Diagram	Chapter 4	24
Figura 6	Homepage – Public Event Discovery Interface	Chapter 5	28
Figura 7	Studio Dashboard – Revenue Analytics and Per-Event Breakdown	Chapter 5	29

Anexa 2 - List of Tables

Table	Caption	Chapter	Page
Tabel 1	Principle entities	Chapter 4	26
Tabel 2	API Response Time Summary	Chapter 6	33
Tabel 3	REST API Endpoint Summary	Appendix4	42

Anexa 3 - List of Acronyms

Acronym	Full Form
ACID	Atomicity, Consistency, Isolation, Durability

API	Application Programming Interface
ASE	Academia de Studii Economice din București
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
CV	Curriculum Vitae
DX	Developer Experience
FOMO	Fear of Missing Out
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IEEE	Institute of Electrical and Electronics Engineers
JWT	JSON Web Token
JSX	JavaScript XML
MVC	Model-View-Controller
NPM	Node Package Manager
ORM	Object-Relational Mapping
PCI-DSS	Payment Card Industry Data Security Standard
QR	Quick Response (code)
RAD	Rapid Application Development
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
WebRTC	Web Real-Time Communication

Anexa 4 - REST API Endpoints Summary

Meth od	Endpoint	Access	Description
Authentication			
POST	/auth/register	Public	Register a new account. Role: DANCER, STUDIO, or AGENCY.
POST	/auth/login	Public	Authenticate with email/password. Returns signed JWT.
Events – Public			
GET	/events	Public	Paginated event listing. Supports filters: query, city, date range, price range.
GET	/events/popular	Public	Top 10 events ranked by weighted score (ticket sales × recency).
GET	/events/nearby	Public	Events within a radius (km) of given coordinates, sorted by distance.
GET	/events/:id	Public	Full details for a single event including creator info and ticket count.
Events – Studio / Agency Management			
POST	/events	STUDIO / AGENCY	Create a new event. Price stored as immutable snapshot at creation time.
PUT	/events/:id	STUDIO / AGENCY	Update event fields. Restricted to the original creator.

DELETE	/events/:id	STUDIO / AGENCY	Delete event. Restricted to the original creator.
POST	/events/:id/invite	STUDIO / AGENCY	Invite a dancer or studio to an event. Triggers in-app notification.
GET	/events/:id/suggested-dancers	STUDIO / AGENCY	Returns dancers whose dance styles match the event's styles.
GET	/events/me/wishlist-analytics	STUDIO / AGENCY	Per-event wishlist save/conversion funnel analytics.
Tickets			
POST	/events/:id/tickets	DANCER	Purchase ticket. Applies dynamic pricing (+15% if >50% sold), 10% commission, loyalty points.
GET	/events/me/tickets	DANCER	Retrieve all tickets owned by the authenticated dancer.
POST	/events/tickets/:id/cancel	DANCER	Cancel a ticket. Refund percentage depends on time remaining before event.
Wishlist			
POST	/events/:id/save	Auth	Save event to personal wishlist.
DELETE	/events/:id/save	Auth	Remove event from wishlist.
GET	/events/me/saved	Auth	Retrieve own wishlist.
GET	/events/:id/saved	Auth	Check whether a specific event is in the authenticated user's wishlist.
Payments			
POST	/payments/intent	DANCER	Create a Stripe PaymentIntent before finalising ticket purchase.
Users & Profiles			
GET	/users/me	Auth	Own full profile including follower counts and loyalty account.
GET	/users/me/loyalty	Auth	Loyalty points balance and last 20 transaction records.

GET	/users/:id	Public	Public profile of any user.
GET	/users/:id/followers	Public	List users who follow the given user.
GET	/users/:id/following	Public	List users the given user is following.
POST	/users/:id/follow	Auth	Follow a user.
DELETE	/users/:id/follow	Auth	Unfollow a user.
GET	/users/search	Auth	Search users by role and name. Used for event invite lookup.
Messages			
POST	/messages	Auth	Send a direct message to another user.
GET	/messages	Auth	Inbox — received messages, newest first.
GET	/messages/sent	Auth	Sent messages, newest first.
PUT	/messages/:id/read	Auth	Mark a received message as read. Only the receiver may call this.
Studio – Agency Collaboration			
POST	/studios/me/collaborations	STUDIO	Create a collaboration request with an agency, optionally linked to an event.